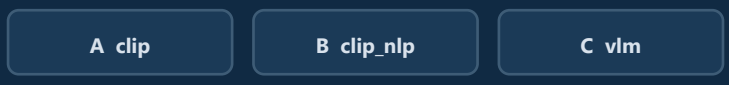
SYSTEM DOCUMENTATION · EVALUATION

Evaluation Architecture and Benchmark Pipeline

How the 240-query Meme-Cap benchmark is drafted, rewritten, leakage-gated, frozen, executed and reported for three retrieval strategies.



PROJECT	Meme-Cap text-to-image retrieval · COMP5523 · repository V2
SOURCE	Code at commit d852b7c · local artifacts generated 1 November 2026
DOCUMENT	Version 1.0 · 4 November 2026



Every statement in this document traces to the code paths and artifacts named in its text.

Contents on the following page

Contents

1 Executive summary . . . 3
1.1 At a glance . . . 3
1.2 The five properties that make the comparison defensible . . . 3
2 System architecture and boundaries . . . 4
2.1 Three evaluation modules, three responsibilities . . . 4
2.2 The identity contract between retrieval and evaluation . . . 5
2.3 Readiness is reported, not assumed . . . 5
3 Corpus, splits and category eligibility . . . 5
3.1 Category eligibility . . . 5
3.2 Why the source split matters . . . 6
4 Benchmark construction: draft → rewrite → gate → freeze . . . 6
4.1 Stage 1 — drafting the review sources . . . 6
4.2 Stage 2 — the reviewed rewrite . . . 6
4.3 Stage 3 — freezing the benchmark . . . 7
4.4 The leakage gate in detail . . . 8
4.5 Stage 4 — graded relevance (qrels) . . . 8
4.6 Artifacts produced by the construction stages . . . 9
4.7 The raw-caption diagnostic set . . . 9
5 Execution: how a frozen query becomes a result . . . 9
5.1 Strategy loading and availability . . . 10
5.2 One retrieval run, two consumers . . . 10
5.3 Retrieval depth . . . 10
5.4 Grouping and availability in the report . . . 10
6 Metrics: definitions and aggregation . . . 10
6.1 Why mAP and recall@R are reported alongside the @k cuts . . . 11
6.2 Aggregation and grouping . . . 11
6.3 The singleton-relevance caveat . . . 11
7 Statistical comparison . . . 11
7.1 Bootstrap confidence intervals . . . 12
7.2 Paired bootstrap test . . . 12
7.3 Holm–Bonferroni across one family . . . 12
7.4 Win / tie / loss . . . 12
7.5 Ablations . . . 12
7.6 What the statistics do not establish . . . 13
8 Reporting and serving the evaluation . . . 13
8.1 The /evaluation page . . . 14
8.2 The probe block . . . 14
9 Reproducibility, determinism and configuration . . . 14
9.1 Configuration knobs . . . 14
9.2 What invalidates what . . . 15
9.3 Tests that protect the evaluation code . . . 15
10 Audit notes: what a careful reader should check . . . 15
A Appendix A — Glossary and file map . . . 16
A.1 Glossary . . . 16
A.2 Where each responsibility lives . . . 16
B Appendix B — One query, end to end . . . 16
C Appendix C — Current local snapshot . . . 17
D Appendix D — Command reference . . . 18

Scope of this document

The reader is assumed to know information retrieval and basic statistics, but not this codebase. Sections 1–3 give the map; Sections 4–8 follow one query from raw annotation to a table cell on the /evaluation page; Sections 9–10 cover reproducibility and the points an auditor should check. Appendix A is the file/function index, Appendix B a worked example, Appendix C the current local snapshot, and Appendix D the command reference.

1 Executive summary

The system retrieves memes from one gallery of 6,378 images using three strategies that differ only in which text they match against the image: **A** `clip` uses the OpenCLIP image embedding alone; **B** `clip_nlp` fuses CLIP with a dense index over human annotations and BM25, with an optional cross-encoder rerank; **C** `vlm` uses offline Qwen2.5-VL descriptions so that no human annotation is needed at serving time. Evaluation exists to answer two separate questions, and the code keeps them separate: *is the benchmark trustworthy?* and *is the difference between strategies real?*

The first question dominates the design. Strategy B indexes the same human annotations that a query could accidentally copy, so a benchmark built naively from captions would reward string overlap rather than retrieval. The pipeline therefore drafts queries, rewrites them away from the annotation wording, runs a lexical leakage gate over the source image and the entire gallery, and freezes the survivors with rule-derived graded relevance. The second question is handled with paired bootstrap tests over per-query metric values and a single Holm–Bonferroni correction across every pairwise comparison.

1.1 At a glance

Item	Value	Where it is defined or stored
Query set	240 queries: 80 visual, 80 intent, 80 metaphor	configs/default.yaml · eval.per_category
Source split	558 test-split images (out of 6,378 gallery images)	SQLite images.split · local store
Distinct query sources	240 (one per query)	data/eval/memecap_eval_v1.json
Graded judgments	339: 240 grade 2 + 99 grade 1	data/eval/memecap_eval_v1.qrels.jsonl
Frozen set	memecap_eval_v1 · never edited after freeze	data/eval/memecap_eval_v1.json
Metrics	P@1/5/10, R@1/5/10, mAP, MRR, nDCG@10, recall@R, median rank, misses	evaluate.py · evaluate_queries
Significance	1,000-sample paired bootstrap, seed 42; Holm across 15 tests	configs/default.yaml · eval.bootstrap/seed
Served report	`compare.json`	data/reports/compare.json

Values measured from the local store and the frozen artifacts on 1 November 2026. “Where it is defined” names the authoritative source, not a copy.

1.2 The five properties that make the comparison defensible

- **Test-only sources.** Every query is written for an image in the 558-image test split, and the freeze verifies that the source is still in that split. The gallery itself remains the full 6,378 images; the split constrains where queries come from, not where retrieval searches.
- **A frozen, auditable set.** Queries and qrels are written once. Each record carries the leakage verdict that admitted it, so the set can be audited without rerunning the gate. A rejected rewrite aborts the build instead of shrinking the set silently.
- **Deterministic relevance.** The source image is grade 2. Grade 1 exists only where a fixed rule fires: a near-identical scene caption, an identical intent sentence, or a shared non-literal metaphor with the same meaning and a document frequency below 10. No model judges relevance.
- **A fair, paired comparison.** All strategies see the same frozen queries against the same gallery. The same cached retrieval run supplies both the headline metrics and the per-query vectors used for paired tests, so a strategy cannot benefit from a lucky second run.
- **Static, traceable reporting.** `tsr compare` writes JSON and HTML once; the web page reads those files. The two header values “Evaluation queries 240” and “Eval set memecap_eval_v1” are fields of `compare.json`, not a live query.

Where the numbers on the /evaluation page come from

The page calls **GET /api/eval/summary**, which returns `data/reports/compare.json` through the `EvalSummaryResponse` schema. The header row is rendered by `web/eval.js`: `report.num_queries` → "Evaluation queries", and `report.eval_set` → "Eval set". `num_queries` is `len(queries)` at compare time; `eval_set` is the query file stem (`memecap_eval_v1`), defaulting to `eval.set_name`. If the page and the artifacts disagree, the page is stale: rerun `tsr compare`.

2 System architecture and boundaries

Evaluation is a layer on top of the same gallery and the same strategy objects that serve the API. Nothing in the evaluation path re-implements retrieval: a strategy is asked for a ranked list, and the evaluator compares that list to qrels. This keeps the measured system and the served system identical, which is the point of evaluating in the first place.



Figure 1. End-to-end architecture. The benchmark layer (04) sits beside the index layer (03): it consumes the same gallery records to build queries and qrels, but never changes an index. The evaluation layer (05) only reads frozen queries and strategy output.

2.1 Three evaluation modules, three responsibilities

Module	Owns	Does not own
`evaluate.py`	Query/qrels data model, leakage checks, drafting and freezing, metric primitives, bootstrap and Holm helpers.	Strategy loading, report layout, HTTP.
`compare.py`	Running strategies over the frozen set, caching retrieval, per-query values, pairwise tests, ablations, report writing.	Building the query set; serving the report.
`cli.py`	The `make-eval`, `eval` and `compare` entry points, label rules for diagnostic sets.	Metric definitions or statistical tests.

2.2 The identity contract between retrieval and evaluation

Qrels are keyed by `image_id`, the stable string from the manifest. Strategy hits carry the SQLite integer primary key. The evaluation runner bridges the two through `store.get(pk).image_id` and caches the mapping, so a qrel can never be matched against the wrong row even if SQLite row order changes. The same `image_id` is the shared key across SQLite, the FAISS id map and `image_ids.npy`; strategy builds may rewrite their own document rows but must never renumber `images`.

2.3 Readiness is reported, not assumed

A strategy that is known but unbuilt returns 503 from the API and is reported as `ready: false` in the evaluation report rather than crashing the run. The evaluator records the load error and continues with the strategies that are available. Strategy C adds a stricter rule: if its description index covers less than 95% of the gallery, it is reported unavailable, so an interrupted captioning run cannot masquerade as a built strategy. This is why a report can legitimately contain two strategies instead of three; the pairwise family is then 1 pair × 5 metrics, not 3 pairs × 5.

> **The gallery is not the test split**
> Retrieval always searches all 6,378 images. The test split (558 images) is the pool from which query sources are drawn, not a filtered retrieval corpus. A common misreading is that evaluation searches only the test split; it does not, and the report's `probe.num_images` confirms the full gallery size.

3 Corpus, splits and category eligibility

Images and annotations come from Meme-Cap (arXiv:2305.13703). The dataset provides 6,382 annotated entries. `tsr prepare` downloads and normalises them, converts PNG to JPEG, and drops duplicate images by SHA-1. The local manifest records 6,378 records, 6,382 annotations and 4 duplicates, so the gallery is 6,378 unique images. The store partitions those images into `test` (558) and `trainval` (5,820).

> **Two documentation numbers are slightly off**
> README.md says "6,382 images" and a "559-image test split"; the generated walkthrough says 558. The local manifest and SQLite store agree on 6,378 unique images and 558 test images. The 6,382 figure is the annotation count before deduplication, and 559 appears to be an off-by-one transcription. Cite 6,378 and 558 when describing the actual gallery, or fix the prose before publication.

3.1 Category eligibility

Each category draws from a different annotation field, so a query is only drafted where the source image can support it:

Category	Eligible when the image has	Draft template	What the query tests
visual	at least one `img_captions` entry	"i am looking for an image that depicts …"	Literal scene description, no intended meaning.
intent	at least one `meme_captions` entry	"find a meme whose message is that …"	The joke or argument the poster conveys.
metaphor	a non-literal metaphor row (label + meaning)	"a meme using … as a symbol for …"	Symbol → intended meaning, not the literal scene.

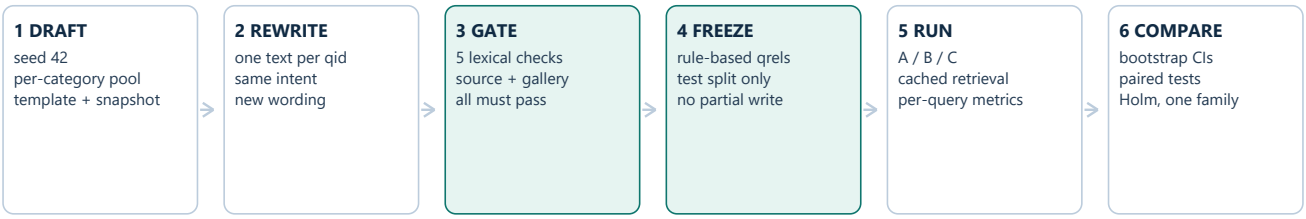
A metaphor row is non-literal when both the label and the meaning are non-empty and the meaning is not the dataset's literal marker "`not related to the meme context`", nor a bare description of the annotation such as "Meme poster". Those rows are still indexed by Strategy B, so they participate in the leakage gate, but they never seed a metaphor query or earn grade-1 relevance. The exclusion is deliberate: "a man" is a literal depiction, not shared symbolism.

3.2 Why the source split matters

The benchmark is documented as test-only, and the freeze rejects any source whose live split is not `test`. This makes the query sample a fixed, documented subset of the corpus rather than an arbitrary draw from images that also appear in training material. It does *not* partition retrieval: the gallery still contains trainval and test images, and Strategy B indexes annotations for both. Leakage control therefore has to be lexical and per-query (Section 4.4), not a split barrier.

4 Benchmark construction: draft → rewrite → gate → freeze

The benchmark is not generated in one shot. It moves through four stages, each with a different failure mode and a different guarantee. The first stage is deterministic and cheap; the second is the only stage that adds new language; the third is the gate; the fourth fixes the qrels and writes the artifacts.



Any rejected rewrite aborts the freeze: the set is never silently shrunk.
The gate is lexical, not semantic: a paraphrase with no shared n-gram passes by design.

Figure 2. The benchmark pipeline. The gate (stage 3) and freeze (stage 4) are the two points where a bad query can stop the build. Everything downstream reads the frozen file and never re-drafts.

4.1 Stage 1 — drafting the review sources

`tsr make-eval` without `--rewrites` calls `draft_eval_set()`. It takes the test split (or the whole gallery if the split has fewer images than requested), builds one candidate pool per category, and shuffles each pool with a local `random.Random(seed)`. Separate pools are important: a category with many rejected drafts must not consume candidates needed by the next category. A global `used` set prevents the same image from seeding two queries.

The draft query is a template around a lightly mangled annotation. `_paraphrase()` applies a small synonym table and swaps common function words (“the” → “this”, “with” → “alongside”), then strips punctuation. It is deliberately crude. The draft is a *starting point for review*, not a benchmark query, and the code says so in the docstring.

The output is `data/eval/draft_sources.json`: one record per qid with `image_id`, `category`, the annotation snapshot (captions, metaphors, title), but **no final query text**. qids are assigned in category order: q0001–q0080 visual, q0081–q0160 intent, q0161–q0240 metaphor (given the YAML order). The snapshot is what makes the later freeze able to detect stale sources.

4.2 Stage 2 — the reviewed rewrite

For every draft qid, a new query text is written and stored as a `qid → text` mapping in `data/eval/rewrites/batch1.json` … `batch6.json` (40 qids each, 240 total). The rewrite is the actual query used in evaluation; the draft text never reaches a strategy.

The rewrite must do four things at once:

- **Preserve the target.** The text must still identify the same source meme and the same category of intent (scene, message, or metaphor). Otherwise the grade-2 qrel would point at a different image than the query describes.
- **Read like a query.** A natural search phrase, not the dataset's annotation register (“Meme poster is conveying that …”).
- **Not borrow annotation wording.** This is the leakage constraint, and it shapes the vocabulary of every rewrite.
- **Stay within the category.** A visual query describes the scene without giving away the intended meaning; an intent query states the message without reciting the scene; a metaphor query names the symbol and its meaning.

Category / qid	Source annotation (abridged)	Frozen rewrite
visual · q0001	"God is holding up a wine cup."	"a robed ancient man with a halo clutching a small cup of wine"
intent · q0081	"Meme poster is annoyed with anti-vaxxer logic."	"a joke about how refusing a vaccine because it is not flawless is like refusing to cross a strong bridge and choosing to swim through rough water instead"
metaphor · q0161	metaphor "a cartoon figure"; meaning "Meme poster trying to explain the male psyche"	"using a cartoon character sprinting into a tunnel to represent men chasing any scrap of affection"

Table 4.1. Three real rewrites from the frozen set, one per category.

Who actually wrote the rewrites? The artifacts do not say.

The code labels every record `query_origin: "llm_rewrite"` — that is the default of `reviewed_eval_set()` and all 240 records carry it. The CLI help calls the directory "reviewed query rewrites" and "written for human review", while the generated walkthrough says each draft is "rewritten by hand". There is no prompt log, model version, or reviewer sign-off in the repository. The defensible description is "reviewed rewrites": text written per draft and accepted through the same gate as any other query. If a report needs a claim about authorship, record it; do not infer it from the label.

Two things the rewrite is not

It is not `_paraphrase()`, the automatic synonym/stopword swap used to produce drafts. And it is not the runtime query expansion in `nlp/analyze.py`: that expansion runs at search time, feeds BM25 only, and never touches the frozen benchmark. Confusing the three is the most common source of "but the query was rewritten" misunderstandings.

4.3 Stage 3 — freezing the benchmark

`tsr make-eval --sources data/eval/draft_sources.json --rewrites data/eval/rewrites --force` calls `reviewed_eval_set()`. It merges the batch files, then processes sources in order. For each source it requires a non-empty rewrite, a known image, a live `test` split, an unchanged annotation snapshot, and a query that passes the leakage gate. If any source fails, the command prints every rejection reason and exits without writing anything.

The snapshot check is a prefix check: the draft stored a prefix of each caption list and the non-literal metaphor rows; the current record must still start with exactly those values (only fields present in the draft are compared). Appending a caption therefore keeps a reviewed query valid, while editing or removing one invalidates it, because the query would now describe text the gallery no longer says. This is what makes "the qid still points at the same evidence" checkable months later.

The rejection vocabulary is small and explicit:

```
missing_rewrite   no text for this qid in any batch file
unknown_image     image_id no longer present in the gallery
not_test_split    the source's live split is trainval
stale_source      the annotation snapshot no longer matches
                  (build_eval_query returns None when leakage fails)
```

On success it writes three files: `memecap_eval_v1.json` (queries, qrels and metadata), `memecap_eval_v1.qrels.jsonl` (one judgment per line), and `memecap_eval_v1.raw_captions.json` (the leaky diagnostic, Section 4.7). Each query record carries `metadata.split`, `metadata.image_id`, `metadata.query_origin` and the full `metadata.leakage` verdict.

4.4 The leakage gate in detail

The gate is the core of the benchmark's credibility. It answers one question: does the query reuse text that a strategy could have indexed? It compares the query against two sets of documents: the source image's own annotation fields, and every annotation in the gallery. Both matter. The source check stops a query from copying the evidence for its own qrel; the gallery check stops a query from copying any other image's annotation, which would create an accidental lexical match elsewhere.

The source documents are exactly what a strategy could index for that image:

```
img_captions[*]  +  meme_captions[*]  +  "<metaphor> means <meaning>"[*]  +  title
```

A query passes only if **all five** checks pass. Each check targets a blind spot in the others.

Check	Rule	Reject when	Why it exists
Source n-gram	8-gram token overlap with the source documents	any shared 8-gram	Catches a long verbatim phrase.
Gallery n-gram	8-gram overlap with all gallery annotations	any shared 8-gram	Catches copying another image's annotation.
Short-doc containment	whole short annotation (4–7 tokens) inside the query	contained verbatim	The 8-gram rule cannot see a 3-word caption pasted whole.
Longest source span	longest contiguous token run shared with the source	≥ 5 tokens	The 8-gram rule misses a 5–7 token copy inside a longer sentence.
Token Jaccard	set overlap with any source document	≥ 0.60	Length-independent; stops short captions being pasted with stopword edits.

Table 4.2. The five leakage checks. Thresholds come from configs/default.yaml: max_ngram_overlap 8, min_copy_tokens 5, Jaccard 0.6 (the Jaccard constant is in check_leakage).

Every verdict is stored, not just the boolean: `max_ngram_overlap`, `strict_overlap`, `contained`, `gallery_contained`, `max_source_span`, `min_copy_tokens`, `jaccard`, `num_tokens`, `passed`, `checked_n`. An auditor can recompute any of them from the frozen query and the manifest without trusting the build.

> **What the gate proves, and what it does not**
>
> It proves the query does not lexically copy an indexed annotation. It does **not** prove the query is semantically original: a paraphrase with no shared 8-gram, no 5-token run and Jaccard below 0.6 can still restate the annotation's meaning. That is accepted deliberately — meaning must be preserved for the qrel to make sense — and the strict thresholds are chosen so the query cannot be answered by string matching alone. The gate is a lexical floor, not a plagiarism detector.

The current frozen set is comfortable inside every threshold: all 240 queries pass; the largest Jaccard is 0.50 (limit 0.60); the longest shared source span is 4 tokens (limit 5); query length ranges from 10 to 35 tokens, median 17.

4.5 Stage 4 — graded relevance (qrels)

Relevance is built by fixed rules, never by a model's judgement. The source image is grade 2. Grade 1 is assigned to another image only when one of three category-specific rules fires:

Category	Grade-1 rule	Implementation detail
visual	Near-identical image caption.	Token Jaccard between the two `img_captions[0]` values ≥ 0.8.
intent	Identical intended meaning.	Exact equality of normalised `meme_captions[0]` (strip + lower).
metaphor	Shared specific metaphor with the same meaning.	Same lower-cased label, meaning sets equal or Jaccard ≥ 0.6, and label document frequency ≤ 10 (`eval.metaphor_max_df`). Literal meanings excluded.

Only the first caption is used for the visual and intent rules; metaphors use every non-literal row. The metaphor rule is deliberately the most restrictive: sharing a label is not enough, because labels such as “a man” or “having” are literal depictions rather than shared symbolism. The label must also be rare (document frequency ≤ 10) and carry the same meaning in both images.

The qrels file is one judgment per line:

```
{"qid": "q0001", "image_id": "memecap_memes_yuirix", "grade": 2}
```

Across the current 240 queries there are 339 judgments: 240 grade 2 and 99 grade 1. 225 queries have only their source image; the other 15 have between 2 and 19 relevant images (three metaphor queries reach 19). This distribution is a property of the corpus, not a bug, but it matters for metric interpretation and is discussed in Section 6.3.

4.6 Artifacts produced by the construction stages

Artifact	Written by	Contents
`data/eval/draft_sources.json`	make-eval (draft pass)	240 qids, source image ids, categories, annotation snapshots; no query text.
`data/eval/batches/batch1-6.json`	review packaging	The same sources split into six 40-item review batches.
`data/eval/rewrites/batch1-6.json`	review	qid → final query text; merged by `_load_rewrites()`.
`data/eval/memecap_eval_v1.json`	make-eval (freeze pass)	Frozen queries with relevant ids, grades, split, origin and leakage verdict.
`data/eval/memecap_eval_v1.qrels.jsonl`	make-eval	One `{qid, image_id, grade}` per line.
`data/eval/memecap_eval_v1.raw_captions.json`	make-eval	Diagnostic set: the same qrels with raw image captions as queries.

All evaluation data lives under data/, which is git-ignored. The artifacts are local; the code that rebuilds them deterministically is versioned.

4.7 The raw-caption diagnostic set

After freezing, `make-eval` builds a second query set by replacing each query with the source image's first raw caption (`img_captions[0]`). It keeps the frozen qrels, marks `query_origin: "raw_caption_diagnostic"`, and recomputes the leakage verdict. This set is **leaky by construction**: Strategy B indexes those exact captions, so its score is an upper bound on annotation-overlap exploitation, not a retrieval result.

The separation is enforced at several points. `tsr eval --queries …raw_captions.json` writes `reports/eval_<key>_raw_captions.json` instead of overwriting the main report. `tsr compare` refuses a diagnostic query file outright. `load_diagnostics()` only folds a diagnostic file into the report if its `eval_set` is the expected `<main>.raw_captions` and its query count matches the main table, so a stale file cannot be compared against a different benchmark.

The diagnostic is useful precisely because it is leaky: it shows how much a strategy would gain if queries reused annotation wording. In the current snapshot, B rises from mAP 0.7608 on the main set to 0.8998 on raw captions, while A and C fall. That contrast is the point of the diagnostic, and it is why the diagnostic must never be merged into the main table.

5 Execution: how a frozen query becomes a result

There are two entry points. `tsr eval` evaluates the selected strategies and writes one `reports/eval_<key>.json` per strategy; it is the tool for diagnostic query sets. `tsr compare` evaluates every available strategy on the frozen set and writes the combined `compare.json`, `compare.html` and `failures.html`. The web page reads only the combined report.

```
python -m tsr make-eval --sources data/eval/draft_sources.json \
                        --rewrites data/eval/rewrites --force
python -m tsr eval --strategy all                # per-strategy JSON
python -m tsr compare --ablations --boot 1000    # combined report + tests
python -m tsr eval --queries data/eval/memecap_eval_v1.raw_captions.json
                                                 # diagnostic only
```

5.1 Strategy loading and availability

`StrategyManager` loads shared resources lazily. In evaluation the strategy is asked to `ensure_loaded()`, which serialises concurrent callers and then reports `ready`. A strategy that cannot load is recorded with its error and an empty metric set; the other strategies still run. This matters for C: its text encoder is available on the serving host even when the captioning GPU is not, so an index can be present and the strategy ready. Conversely, a partially captioned gallery is rejected by the 95% coverage rule.

5.2 One retrieval run, two consumers

The runner wraps each strategy in `_CachedRetriever`. The cache key is `(query_text, k)`, and the same object is passed to both `evaluate_queries()` (which computes the headline metrics and the per-query rows) and `per_query_values()` (which produces the vectors the bootstrap needs). The second call hits the cache, so each query is searched once per strategy. This is not just an optimisation: it guarantees that the paired test is computed on the same ranked lists that produced the headline number. If a strategy raises, the runner logs it, counts an error, caches an empty list, and keeps going; the `num_errors` field records how often that happened.

5.3 Retrieval depth

The evaluator does not retrieve only `max(top_k)`. It uses:

```
depth = max(max(top_k), largest relevant set, 100)
```

With top_k = 1/5/10, the depth is at least 100. This prevents recall@R and mAP from being truncated when a query has more relevant images than 10 — the largest current relevant set is 19 — and gives AP a stable denominator. The cost is one deep search per query per strategy, which is why the cache matters.

The returned list is mapped from SQLite primary keys to `image_id` values and compared with the qrels set. Per query, the evaluator stores the rank of the first relevant hit, AP, RR, nDCG@10 and the full retrieved list; these row-level records are what `failures.html` and the per-query bootstrap consume.

5.4 Grouping and availability in the report

Every query contributes to two groupings: its category (visual / intent / metaphor) and its split (test). The report stores `per_category` and `per_split` with count, mAP and MRR. Because the benchmark is test-only, the split table has a single row; the grouping still exists so the report format does not change if a future set includes another split. The web page renders both tables and shows a dash rather than a fabricated row when a group is absent.

6 Metrics: definitions and aggregation

All metrics are computed per query and then macro-averaged over the query set. Let R be the set of relevant image ids (grades 1 and 2), g(d) the grade of document d, and π the ranked list returned by the strategy.

Metric	Definition	Notes
P@k	number of d in π[:k] that are in R, divided by k	Denominator is k, even if fewer than k results exist.
R@k	number of d in π[:k] that are in R, divided by |R|	Saturates when |R| < k; read with recall@R.
AP	(1 / min(|R|, |π|)) x sum over hits i of hits(i)/i	With depth ≥ 100 and |R| ≤ 19, the denominator is |R|.
RR	1 / rank of the first relevant hit, else 0	Mean over queries is MRR.
nDCG@k	DCG@k / IDCG@k, gain = 2^g - 1, discount = log2(i+1)	Graded; grade 2 contributes 3x grade 1.
recall@R	number of d in π[:|R|] that are in R, divided by |R|	Not truncated by the reported @k cuts.
median rank	median first-relevant rank over queries with a hit	Misses are excluded from the median.
misses	count of queries with no relevant hit in the retrieved depth	Reported as a count, not a rate.

Table 6.1. Metric primitives in evaluate.py. The aggregation is a simple mean over queries unless stated.

The headline table also reports bootstrap confidence intervals for mAP, MRR and nDCG@10, and p50/p95 search latency. Latency is measured around `strategy.search()` only, on the host that ran compare. It is useful for spotting a pathological cost (B's cross-encoder rerank, for example) but it is not a controlled serving benchmark: hardware, thread pinning, cache warmth and concurrent load all move it.

6.1 Why mAP and recall@R are reported alongside the @k cuts

P@k and R@k at 1/5/10 are familiar but easy to misread when most queries have one relevant image. For a singleton query, P@1 is 1 only if the source is ranked first, R@10 is 1 if it appears anywhere in the top 10, and mAP is 1/rank. The three views disagree by design. Reporting mAP and recall@R as well makes the rank distribution visible instead of collapsing it into a single cut.

6.2 Aggregation and grouping

Each metric is averaged over all queries; the report additionally stores per-category and per-split means for mAP and MRR. Per-category means are unweighted (each category has 80 queries in the current set, so they also sum to the overall mean). The per-query rows are retained in `strategies.<key>.per_query`, which is what allows failures.html and the paired bootstrap to work without re-running retrieval.

6.3 The singleton-relevance caveat

225 of 240 queries have exactly one relevant image. For those, mAP equals reciprocal rank, and the difference between mAP and MRR is entirely driven by the 15 queries with grade-1 neighbours. This is not a flaw in the metric code; it is the shape of the benchmark. It does mean that small mAP differences on the main table should be read as "rank of the source image" differences, and that the metaphor category (where grade-1 sets are larger) carries more of the graded-relevance signal than its 80-query share suggests.

7 Statistical comparison

The comparison asks whether one strategy is better than another on the same 240 queries. Because the queries are shared, the right unit is the per-query difference, not two independent means. The code implements three layers: bootstrap confidence intervals for each strategy, a paired bootstrap test for each pair and metric, and one Holm–Bonferroni correction across the whole family.

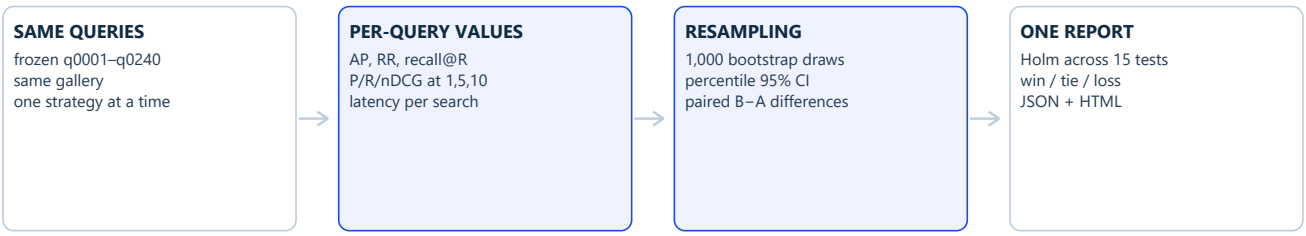


Figure 3. From retrieval to inference. The per-query metric vectors produced during the cached run feed both the confidence intervals and the paired tests; the report is a snapshot of that one run.

7.1 Bootstrap confidence intervals

`bootstrap_ci()` resamples the n per-query values with replacement `n_boot` times (default 1,000), takes the mean of each resample, and reports the 2.5th and 97.5th percentiles around the observed mean. The seed is fixed (42 by default), so the intervals are reproducible. The interval describes uncertainty over the query sample, not over hardware or model weights.

7.2 Paired bootstrap test

For strategies A and B, `paired_bootstrap_test()` forms the per-query difference `d_i = B_i − A_i`, resamples the differences 1,000 times, and reports the mean delta with a 95% percentile interval and a two-sided p-value:

```
p = 2 x min( P(boot ≤ 0), P(boot ≥ 0) )
p = max(p, 1 / n_boot)
```

The floor matters. A finite bootstrap cannot resolve a p-value below 1/n_boot; reporting 0 would overclaim certainty. With 1,000 resamples the smallest reportable p is 0.001, and after Holm the smallest adjusted value is still bounded by that resolution. The current report's significant comparisons show p_holm = 0.015, which is the Holm-adjusted value of the 0.001 floor across the family (see below), not a typo.

7.3 Holm–Bonferroni across one family

The five compared metrics are mAP, MRR, nDCG@10, P@1 and R@10. With three ready strategies there are three pairs × five metrics = 15 tests. Correcting separately inside each pair would treat the three comparisons as independent families; the report claims one family, so `holm_bonferroni()` is applied once to all 15 raw p-values. The adjusted value is the running maximum of `(m − i) · p_(i)` in sorted order, capped at 1. A comparison is marked significant when `p_holm < 0.05`.

Holm controls the family-wise error rate without assuming independence, which is the right trade-off here: the five metrics are correlated (they are computed from the same ranked lists), and a Bonferroni correction would be needlessly conservative, while no correction would inflate the chance of a false positive across 15 tests.

7.4 Win / tie / loss

Alongside the delta, each comparison stores the per-query win/tie/loss count: B wins a query when its metric is higher, loses when lower, ties within 1e−9. This is descriptive, not a test. It is useful when the mean delta is driven by a few large gains rather than a broad shift, and it appears in the report next to the p-value so a reader can see both magnitude and breadth.

7.5 Ablations

`--ablations` re-instantiates a strategy with a modified configuration, runs the same frozen queries, and records the variant's mAP, CI and config hash. Variants are configuration changes, not retrained models: they isolate which signal or component contributes.

Variant	Change	Current mAP
clip + prompt ensemble	A with prompt ensemble enabled	0.4415
clip_nlp − BM25	B with BM25 weight 0	0.7040
clip_nlp − dense doc	B with dense document weight 0	0.7043
clip_nlp − CLIP	B with CLIP weight 0	0.7802
clip_nlp − reranker	B with cross-encoder rerank disabled	0.6887
vlm only (no CLIP)	C with clip_weight 0	0.3922
vlm + CLIP 0.35	C with clip_weight 0.35	0.4720

Table 7.1. Ablation variants and their mAP in the local snapshot of 1 November 2026. The base B mAP is 0.7608, so removing CLIP improves B slightly in this run; that is a finding to report, not a reason to hide the variant.

An ablation is best-effort: a variant that fails to load is recorded as unavailable and does not abort the comparison. Each successful variant stores `config_hash`, so a reader can tell which configuration produced the number.

7.6 What the statistics do not establish

- The 240 queries are a sample from a 558-image test split, not a random sample of all possible meme-search intents. The CI covers query sampling variability under that construction, not distribution shift.
- The paired test assumes queries are exchangeable. Stratification (80 per category, drawn separately) means the effective sample is not a simple random sample; per-category tables are the honest place to look for category-specific effects.
- Significance is not effect size. A p_holm below 0.05 says the direction is unlikely under the null; the delta and its interval say whether the difference is worth caring about.
- The bootstrap resolution floor means very small p-values are not distinguishable; do not report p < 0.001 as if it were exact.

8 Reporting and serving the evaluation

`compare_strategies()` assembles one report dictionary and writes three files: `compare.json` (machine-readable, the source of truth), `compare.html` (a self-contained human report) and `failures.html` (the ten worst queries per strategy, with relevant and top-5 retrieved images side by side). The web app serves the static pages; it does not recompute metrics.

Report field	Meaning
`eval_set`	Query-file stem, e.g. memecap_eval_v1. This is what the page shows as “Eval set”.
`num_queries`	len(queries) at compare time. The page shows this as “Evaluation queries”.
`top_k / n_boot / seed`	Evaluation configuration used; makes the report self-describing.
`strategies.<key>.metrics`	Mean metrics plus median rank, misses and num_queries.
`strategies.<key>.ci`	Bootstrap mean/low/high for each metric.
`strategies.<key>.per_query`	Per-query rank, AP, RR, nDCG@10 and the full retrieved list.
`strategies.<key>.per_category`	mAP and MRR by query category.
`strategies.<key>.per_split`	mAP and MRR by source split (test in the current set).
`pairwise`	Per pair and metric: delta, CI, raw p, Holm p, win/tie/loss.
`ablations`	Variant results keyed by human-readable label.
`diagnostics`	Raw-caption mAP/MRR/nDCG@10, only if the matching diagnostic files exist.
`probe`	Gallery size, leakage pass/flag counts, VLM caption count under provenance filters.
`failures`	Worst queries with pk/path for the HTML strip; not part of the schema.

Table 8.1. compare.json fields. EvalSummaryResponse mirrors every field except failures, which exists only so the HTML failure gallery can link back to images.

8.1 The /evaluation page

The page is a static shell (`web/evaluation.html`) plus `web/eval.js`. On load it fetches `GET /api/eval/summary`; the API reads `data/reports/compare.json` through `load_report()` and returns it as `EvalSummaryResponse`. If the file is missing, the endpoint returns **404** with the instruction to run `tsr compare`; the HTML endpoints `/api/eval/report.html` and `/api/eval/failures.html` behave the same way. The page renders:

- **Overall** — strategy rows with mAP [CI], MRR, nDCG@10, P@1, R@10, median rank and p50/p95 latency; unavailable strategies show the recorded error.
- **By category** — mAP/MRR for visual, intent and metaphor.
- **By split** — the report's per_split table (test).
- **Pairwise** — delta, interval, raw and Holm p, win/tie/loss.
- **Ablations** — variant rows when `--ablations` was used.
- **Diagnostics** — raw-caption mAP next to the main-set mAP, explicitly labelled leaky.
- **Probe** — gallery size, leakage pass/flag counts, VLM coverage.
- **Links** — the full HTML report and the failure gallery.

> **Reports are snapshots, not live results**
> If you change a strategy, rebuild an index, or edit a config, the page keeps showing the old numbers until `tsr compare` runs again. There is no cache invalidation and no version check against the code or index. Always read `generated_at` before quoting the page, and regenerate after any change to a strategy or config.

8.2 The probe block

The report's `probe` is a lightweight consistency check: `num_images` is `store.count()`; `leakage_passed/flagged` count the frozen set's verdicts; `vlm_captions` counts VLM documents under the current provenance filters. If `num_images` is smaller than expected, the index or store is incomplete. If `leakage_flagged` is non-zero, the set is not the frozen one. These are guard rails, not metrics.

9 Reproducibility, determinism and configuration

Three kinds of determinism are at work, and they are independent:

- **Sampling determinism.** `draft_eval_set()` uses `random.Random(seed)` with seed 42, shuffling each category's candidate pool separately. The same manifest and config produce the same draft qids and source images.
- **Statistical determinism.** `bootstrap_ci()` and `paired_bootstrap_test()` use `np.random.default_rng(seed)` with seed 42, so intervals and p-values are reproducible for the same per-query values.
- **Content determinism.** The freeze compares the draft's annotation snapshot with the live record and rejects a stale source; the leakage verdict is stored. A rebuilt set from the same inputs and rewrites has the same content, even if file metadata such as the export timestamp differs.

9.1 Configuration knobs

Option	Default	Role
`eval.set_name`	memecap_eval_v1	Stem for the frozen files and the report's eval_set label.
`eval.num_queries`	240	Default draft size when --num is omitted.
`eval.per_category`	80 / 80 / 80	Stratification; scaled proportionally if --num differs.
`eval.top_k`	[1, 5, 10]	Reported cuts. Retrieval depth is at least 100 regardless.
`eval.seed`	42	Draft sampling and bootstrap seed.
`eval.bootstrap`	1000	Resamples for CIs and paired tests.
`eval.max_ngram_overlap`	8	Leakage n-gram size.

Option	Default	Role
`eval.min_copy_tokens`	5	Longest shared source span allowed (must be < this).
`eval.metaphor_max_df`	10	Shared metaphor labels must be rarer than this to earn grade 1.

Runtime-adjustable knobs (strategy weights, rerank settings) live in `settings.py`; build-time and structural options live in `configs/default.yaml`. Changing either without rerunning compare leaves the report describing the old configuration.

9.2 What invalidates what

Change	Required action
New or edited rewrites / sources	Rerun `make-eval --sources --rewrites --force`, then eval and compare.
Annotation text or caption order	The freeze will reject stale sources; rebuild the drafts or update the snapshot.
Strategy weights, rerank, prompt settings	Rerun compare (and ablations if relevant); no re-freeze needed.
Index rebuild (same manifest)	Rerun eval/compare; qrels and queries are unchanged.
Dataset rebuild (new manifest)	Rebuild everything, including the benchmark; image ids may change.

9.3 Tests that protect the evaluation code

The suite covers the pieces most likely to drift: draft stratification and per-category pools, leakage gating (including short-document containment and stale sources), graded qrels, bootstrap and Holm helpers, report assembly and diagnostic separation. Unit tests use fake encoders and never download a model, so they run in CI without GPU or network. Run:

```
PYTHONPATH=src pytest
PYTHONPATH=src ruff check src tests
```

10 Audit notes: what a careful reader should check

The following are not failures of the pipeline; they are properties a reviewer should know before quoting a number. Each has a concrete implication.

Observation	Implication
1. Documentation counts disagree with the store.	README says 6,382 images and 559 test images; the manifest and SQLite report 6,378 and 558. Fix the prose or state the pre-dedup annotation count explicitly.
2. Rewrite authorship is not recorded.	The tag says llm_rewrite; the docs say "by hand". The gate certifies non-copying, not authorship. Add a provenance note if the report makes a claim about who wrote the queries.
3. The benchmark covers 43% of the test split.	240 distinct source images out of 558. That is a large sample but not the whole split; category-level estimates (80 queries each) are correspondingly noisier than the pooled number.
4. Most queries have one relevant image.	225/240 are singletons, so mAP equals reciprocal rank for them. Graded nDCG is driven by the 15 queries with grade-1 neighbours, especially the metaphor category.
5. The gate is lexical.	A semantic paraphrase with no shared n-gram passes. This is intended, but it means "leakage-free" is a lexical claim, not a semantic guarantee.
6. The diagnostic is not a result.	Raw-caption mAP is an upper bound for annotation overlap. Never put it in the main table; the code refuses to compare on it, and the report labels it separately.
7. Latency is host- and run-specific.	p50/p95 are useful for relative cost, not for a serving SLA. Thread pinning and cache warmth change them.
8. The p-value floor is 0.001.	With 1,000 resamples, p_holm = 0.015 is a family-adjusted floor, not an exact probability. Do not describe it as p < 0.001.

Observation	Implication
9. The report has no code or index version.	generated_at is the only freshness signal. A report generated before a strategy change looks identical to a current one. Consider recording the commit and index meta hash in compare.json.
10. Ablations are configuration variants.	They are not retrained models. "B − CLIP" means the CLIP weight was set to zero, not that a new model was trained without CLIP.

A Appendix A — Glossary and file map

A.1 Glossary

Term	Meaning in this codebase
qid	Query identifier, q0001–q0240, assigned at draft time and preserved through the freeze.
qrels	Graded relevance judgments: one (qid, image_id, grade) row per relevant image.
grade 2 / grade 1	Source image / rule-derived neighbour. No grade 3; no model judge.
leak	A query that shares a lexical span with an indexed annotation (source or gallery).
freeze	The one-way transition from reviewed drafts to the final query and qrel files.
diagnostic	The raw-caption query set, leaky by construction, reported separately.
ablation	A strategy re-instantiated with one configuration change, same queries.
paired bootstrap	Resampling per-query differences to test whether B−A is centered on zero.
Holm	A step-down family-wise correction applied once across all pair × metric tests.

A.2 Where each responsibility lives

Path	Responsibility
`src/tsr/evaluate.py`	EvalQuery model; save/load; metric primitives; bootstrap; leakage checks; drafting; qrels; freezing.
`src/tsr/compare.py`	Strategy runner and cache; per-query values; paired tests; Holm; ablations; report/HTML writers.
`src/tsr/cli.py`	make-eval / eval / compare commands; diagnostic filename labels; refusal rules.
`src/tsr/schemas.py`	EvalSummaryResponse, the shape the API promises the UI.
`src/tsr/api.py`	/api/eval/summary, /api/eval/report.html, /api/eval/failures.html, /evaluation.
`web/eval.js`	Rendering of overall, category, split, pairwise, ablation, diagnostic and probe tables.
`configs/default.yaml`	eval.* knobs: set name, counts, seed, bootstrap, thresholds.
`data/eval/`	Draft sources, review batches, rewrites, frozen queries, qrels, diagnostic.
`data/reports/`	compare.json/html, failures.html, per-strategy eval_.json.
`tests/test_evaluate.py`	Drafting, leakage, qrels, reviewed-set gating.
`tests/test_compare.py`	Report assembly, diagnostics, pairwise behaviour.

B Appendix B — One query, end to end

This is the full life of q0001, a visual query. Every value below is quoted from the local artifacts; it is the clearest way to see how the stages connect.

Stage	What happens	Value
Source	Meme-Cap annotation for `memecap_memes_yuirix` (test split).	img_caption: “God is holding up a wine cup.” meme_caption: “Meme poster is conveying that God gave us all water then put salt in it so we can't drink it.”
Draft	Template around `img_captions[0]`, paraphrase applied; qid assigned.	Stored in draft_sources.json as q0001 / visual; the draft text is a starting point, not the query.
Rewrite	New text written for the qid.	“a robed ancient man with a halo clutching a small cup of wine”
Gate	Five lexical checks against source and gallery.	8-gram overlap 0; strict overlap 0; no contained short doc; max source span 0; Jaccard 0.20. Pass.
Qrels	Grade 2 for the source; grade-1 rules evaluated.	Only `memecap_memes_yuirix` (grade 2). No other image has a caption with Jaccard ≥ 0.8.
Freeze	Query and qrels written; qid preserved.	memecap_eval_v1.json + one qrels line.
Retrieval	A / B / C each return a ranked list at depth ≥ 100.	A: rank 14 · B: rank 2 · C: rank 3
Scoring	AP/RR/nDCG computed from the rank; vectors feed the bootstrap.	A: AP 0.0714, RR 0.0714 · B: AP 0.50, RR 0.50 · C: AP 0.3333, RR 0.3333

For this query the three strategies disagree sharply: B's annotation index places the source second, while CLIP alone places it 14th. Because the query is a singleton, AP is exactly 1/rank; the bootstrap will use those three numbers as one of its 240 paired observations. This is the unit of analysis the whole report is built from.

> **Why q0001 is a useful sanity check**
> The query avoids the annotation's proper noun (“God”) and its joke (“water… salt”). It describes the picture instead — robe, halo, cup — which is exactly what a visual query should do, and it is why A can still retrieve the source at rank 14 despite never seeing the annotation. If the gate were removed, a query copying “God is holding up a wine cup” would flatter B without measuring visual retrieval.

C Appendix C — Current local snapshot

These are the values in `data/reports/compare.json` generated 1 November 2026 at 15:12 UTC. They describe the local artifacts at that moment, not a permanent result. All three strategies were ready; the gallery had 6,378 images and 6,378 VLM captions.

Strategy	P@1	R@10	mAP	MRR	nDCG@10	median	misses	p50 / p95 (ms)
A clip	0.3583	0.6074	0.4395	0.4461	0.4744	2.0	46	14.4 / 20.8
B clip_nlp	0.7000	0.8702	0.7608	0.7647	0.7804	1.0	13	225.6 / 285.9
C vlm	0.3833	0.6259	0.4519	0.4632	0.4881	2.0	38	35.3 / 41.6

Table C.1. Headline metrics for the frozen 240-query set. CIs are in the JSON; the page shows them on hover.

Strategy	visual mAP	intent mAP	metaphor mAP
A clip	0.3153	0.4444	0.5588
B clip_nlp	0.6313	0.7389	0.9121
C vlm	0.2469	0.5390	0.5697

Table C.2. mAP by category. B's advantage is largest on metaphor, where its documents carry the human-written meaning.

Comparison	mAP delta	p (raw)	p (Holm)	Verdict
B − A	+0.3213	0.001	0.015	B significantly better on all five metrics
B − C	+0.3089	0.001	0.015	B significantly better on all five metrics
A − C	-0.025 to -0.012	> 0.05	> 0.05	No significant difference on any of the five

Table C.3. Pairwise summary. The Holm family is 3 pairs × 5 metrics = 15 tests.

Diagnostic (raw captions)	mAP	Main-set mAP	Reading
A clip	0.3546	0.4395	Worse with raw captions; no annotation index to exploit.
B clip_nlp	0.8998	0.7608	Rises sharply: it is matching text it indexed. Leaky upper bound.
C vlm	0.2591	0.4519	Falls; raw captions are a poor query register for the VLM documents.

Table C.4. Diagnostic table. It is deliberately separate and must not be quoted as retrieval quality.

Reading the snapshot responsibly

The gap B−A is large and consistent across metrics, which is what the paired test is designed to detect. The B−C gap is similar in size. A−C is not significant. The ablation table in Section 7.5 shows that removing CLIP from B slightly improves mAP in this run (0.7802 vs 0.7608), so "B = CLIP + NLP" is not a claim that every component helps. Report the components, not the slogan.

D Appendix D — Command reference

```
# Build the gallery and strategy indexes
python -m tsr prepare
python -m tsr build --strategy gallery
python -m tsr build --strategy all

# Freeze the benchmark from reviewed sources
python -m tsr make-eval --sources data/eval/draft_sources.json \
                        --rewrites data/eval/rewrites --force

# Evaluate
python -m tsr eval --strategy clip,clip_nlp,vlm
python -m tsr compare --ablations --boot 1000

# Leaky diagnostic (separate report, never the main table)
python -m tsr eval --queries data/eval/memecap_eval_v1.raw_captions.json

# Serve the report
python -m tsr serve          # http://localhost:8000/evaluation

# Tests and lint
PYTHONPATH=src pytest
PYTHONPATH=src ruff check src tests
```

All commands accept `--config path.yaml` or respect `TSR_CONFIG`; environment overrides use `TSR_` plus the upper-cased dotted path, joined by __. The evaluation artifacts are written under `data/eval/` and `data/reports/`, both ignored by git.

Document version 1.0 · generated 4 November 2026 from commit d852b7c. The PDF is generated by docs/make_evaluation_pdf.py; edit that script, not the PDF.